

Tecnológico de Monterrey

Evidencia 2

Alumno: Diego Javier Martínez Sánchez

Matrícula: A00845422

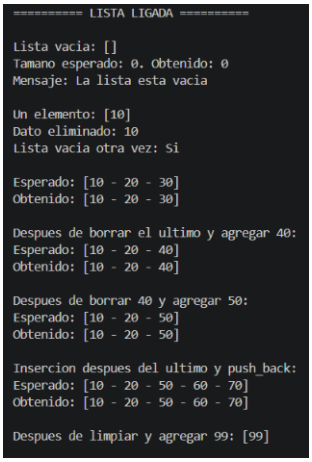
Materia: Estructura de datos y algoritmos fundamentales

Grupo: 204

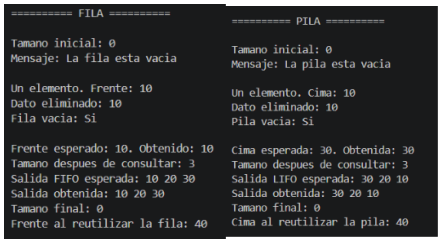
Profesor: David Alonso Cantú Delgado

Fecha de entrega: 18 de octubre de 2026

PRUEBAS



Se probó la lista vacía, la inserción y eliminación de un solo elemento y las operaciones al inicio y al final. Los resultados coincidieron con los esperados. También se comprobó que la lista pudiera utilizarse nuevamente después de limpiarla.



La fila devolvió los datos en el orden 10, 20 y 30, siguiendo el comportamiento FIFO. La pila los devolvió como 30, 20 y 10, siguiendo LIFO. En ambos casos, intentar eliminar un elemento cuando la estructura estaba vacía mostró un mensaje sin detener el programa.

```
========== LISTA DOBLEMENTE LIGADA ==========

Lista vacia: [] (0 elementos)
Recorrido inverso: [] (0 elementos)
Eliminar nodo nulo: Sin cambios

Un elemento: [10] (1 elementos)
Despues de eliminar el unico nodo: [] (0 elementos)
Ambos extremos nulos: Si

Adelante esperado: [10 <-> 20 <-> 30 <-> 40]
Obtenido: [10 <-> 20 <-> 30 <-> 40] (4 elementos)
Atras esperado: [40 <-> 30 <-> 20 <-> 10]
Obtenido: [40 <-> 30 <-> 20 <-> 10] (4 elementos)

Despues de borrar 30:
[10 <-> 20 <-> 40] (3 elementos)
[40 <-> 20 <-> 10] (3 elementos)

Despues de borrar ambos extremos:
Esperado: [20]
[20] (1 elementos)
[20] (1 elementos)

Insercion despues de tail y addLast:
Esperado: [20 <-> 50 <-> 60]
[20 <-> 50 <-> 60] (3 elementos)
[60 <-> 50 <-> 20] (3 elementos)

Despues de limpiar y agregar 99:
[99] (1 elementos)
[99] (1 elementos)
```

Se comprobó el recorrido de la lista hacia adelante y hacia atrás, además de insertar y eliminar nodos en distintas posiciones. Al eliminar el único elemento, ambos extremos quedaron en nulo. Los recorridos siguieron funcionando después de modificar y reutilizar la lista.

```
 -Wextra -I include src/main.cpp src/Log.cpp src/FileManager.cpp -o evidence2.exe
>> if ($?) { .\evidence2.exe }

========== CARGA DE LOGS ==========
1. log607-1.txt
2. log607-2.txt
3. Escribir otro nombre o ruta
0. Salir
Selecciona una opcion: 3
Nombre o ruta del archivo: data/prueba_validacion.txt
Linea 2 omitida: Dia invalido para el mes indicado: 30
Linea 3 omitida: Direccion IP invalida: 10.14.300.232
Linea 4 omitida: La descripcion del evento esta vacia

========== RESUMEN DE CARGA ==========
Archivo: data/prueba_validacion.txt
Registros cargados: 1
Lineas omitidas: 3

Primer registro recibido:
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts

Ultimo registro recibido:
Sep 29 2024 14:37:38 10.14.19.188 Unauthorized Access Attempts

Los registros conservan el orden de llegada.

========== EVIDENCIA 2 ==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Selecciona una opcion:
```

El programa detectó tres líneas inválidas: una con un día inexistente, otra con una IP fuera de rango y otra sin descripción. Mostró el número y motivo de cada error, omitió esas líneas y cargó el registro válido sin detenerse.

```
 -Wextra -I include tests/sort_test.cpp -o tests/sort_test.exe
>> if ($?) { .\tests\sort_test.exe }o\repo estructura\C.MTY.TC1031.607.2613.A00845422\C.MTY.TC1031.607.2613.A00845422\Evidences\Evidence2>
========== PRUEBA DE MERGE SORT ==========

Lista vacia: [] (0 elementos)
Comparaciones: 0
Profundidad: 0

Un elemento: [10] (1 elementos)

Antes de ordenar: [40 <-> 10 <-> 30 <-> 20 <-> 10] (5 elementos)
Orden ascendente: [10 <-> 10 <-> 20 <-> 30 <-> 40] (5 elementos)
Orden descendente: [40 <-> 30 <-> 20 <-> 10 <-> 10] (5 elementos)
Comparaciones: 7
Profundidad maxima: 4

Antes de ordenar los empates: [2A <-> 1B <-> 2C <-> 1D <-> 2E] (5 elementos)
Esperado: [1B <-> 1D <-> 2A <-> 2C <-> 2E]
Obtenido: [1B <-> 1D <-> 2A <-> 2C <-> 2E] (5 elementos)

Todas las pruebas pasaron correctamente.
```

Se probó el ordenamiento con una lista vacía, un elemento y varios datos desordenados. Los valores quedaron ordenados y pudieron recorrerse en ambas direcciones. La prueba con letras también comprobó que los elementos con la misma clave conservaran su orden de llegada.

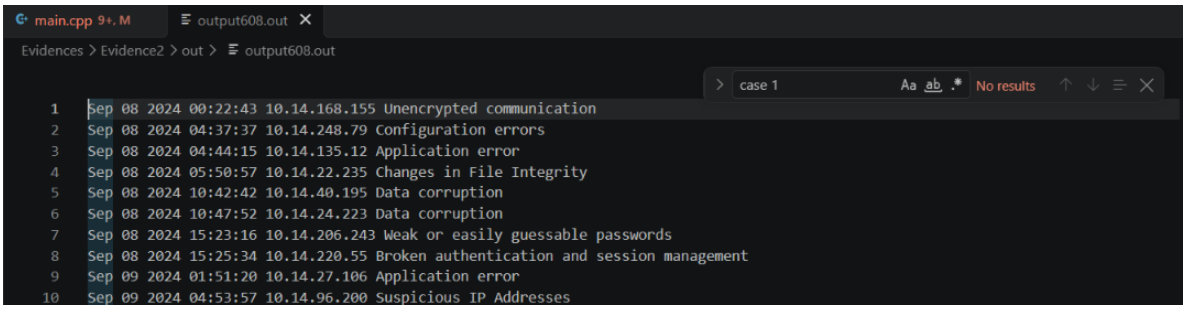
```
========== RESULTADOS DEL ORDENAMIENTO ==========
Algoritmo: Merge Sort
Archivo: data/log607-1.txt
Registros: 6818
Prediccion: n
Tiempo: 1457.3 microsegundos
Comparaciones entre datos: 78330
Profundidad maxima de recursion: 14
Verificacion del orden: Correcto
Crecimiento teorico de Merge Sort: O(n log n)
Coincidio la prediccion con el crecimiento teorico: No
Esta evaluacion usa la complejidad teorica; una sola corrida no demuestra el crecimiento.

Primer registro ordenado:
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

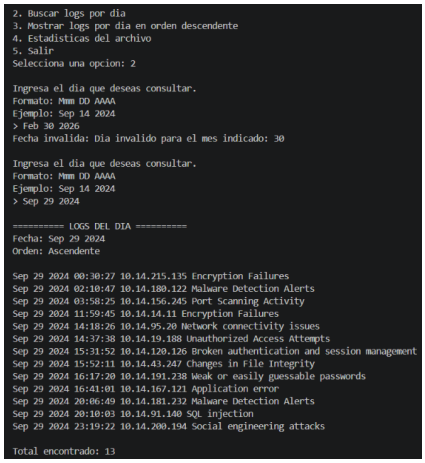
Ultimo registro ordenado:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Archivo generado: out/output608.out
```

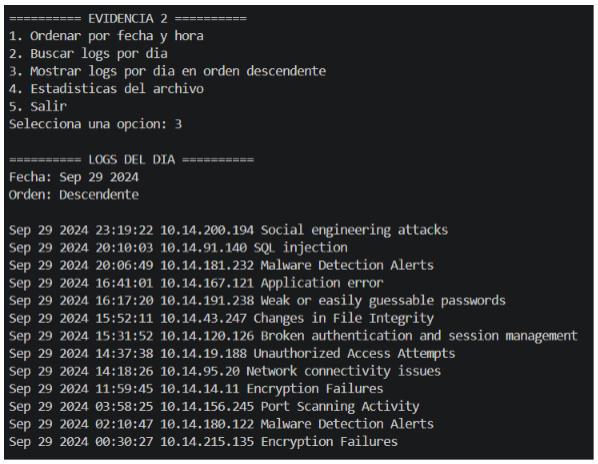
Se ordenaron 6818 registros mediante Merge Sort, con 78330 comparaciones y una profundidad máxima de 14. El programa confirmó el orden y generó el archivo de salida. Mi predicción fue un crecimiento lineal, por lo que no coincidió con el crecimiento teórico del algoritmo.



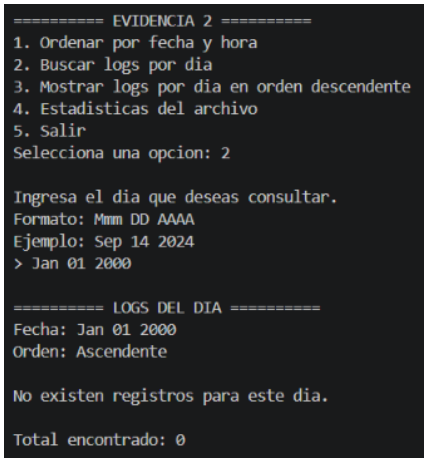
El archivo output608.out conserva el formato de los registros originales. En las primeras líneas se observa que los eventos están ordenados por fecha y hora, comenzando con los del 8 de septiembre de 2024.



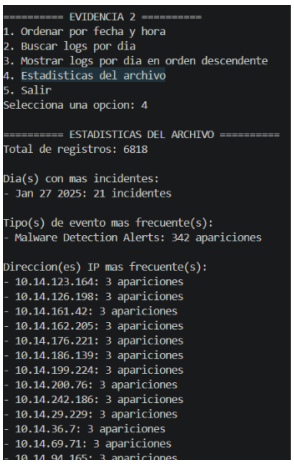
Al ingresar Feb 30 2026, el programa detectó que el día no existe y solicitó otra fecha. Después consulté Sep 29 2024 y se mostraron sus 13 registros en orden de hora, del más antiguo al más reciente.



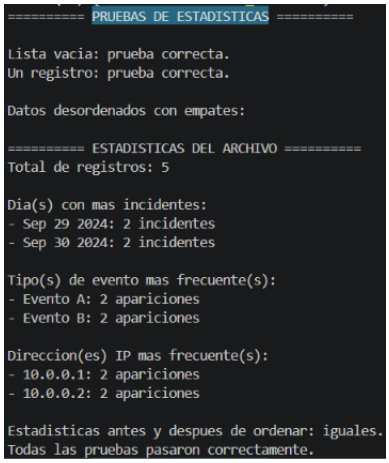
La opción 3 mostró los registros de Sep 29 2024 del más reciente al más antiguo. La pila permitió invertir el orden de los resultados de la búsqueda anterior.



Al consultar Jan 01 2000, el programa indicó que no había registros para ese día y mostró un total de cero, sin detener la aplicación.



Se analizaron 6818 registros. El día con más incidentes fue Jan 27 2025, con 21, y el evento más frecuente fue Malware Detection Alerts, con 342 apariciones. También se mostraron las IP empatadas en la frecuencia máxima de tres apariciones.



Con cinco registros de prueba se comprobaron empates entre días, eventos e IPs. El programa mostró todos los máximos empatados y verificó que los conteos fueran iguales antes y después de ordenar. También pasaron las pruebas con una lista vacía y un solo registro.

```
========== EVIDENCIA 2 ==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Selecciona una opcion: 1

La lista ya esta ordenada. No se ejecutara nuevamente Merge Sort.
Archivo generado: out/output608.out
```

Al intentar ordenar nuevamente, el programa detectó que los registros ya estaban ordenados. Mostró el aviso correspondiente y generó el archivo de salida sin repetir Merge Sort.

```
========== EVIDENCIA 2 ==========
1. Ordenar por fecha y hora
2. Buscar logs por dia
3. Mostrar logs por dia en orden descendente
4. Estadisticas del archivo
5. Salir
Selecciona una opcion: 2

Primero debes ordenar los registros con la opcion 1.
```

Al seleccionar la búsqueda antes de ordenar, el programa indicó que primero debía utilizar la opción 1. Así se evita buscar sobre registros cuyo orden todavía no se ha confirmado.

```
========== CARGA DE LOGS ==========
1. log607-1.txt
2. log607-2.txt
3. Escribir otro nombre o ruta
0. Salir
Selecciona una opcion: 3
Nombre o ruta del archivo: data/no_existe.txt

Error: No se pudo abrir el archivo: data/no_existe.txt
Selecciona nuevamente el archivo.
```

Al ingresar la ruta de un archivo inexistente, el programa mostró que no pudo abrirlo y permitió seleccionar otro archivo, sin cerrar la aplicación.

```
========= RESULTADOS DEL ORDENAMIENTO =========
Algoritmo: Merge Sort
Archivo: data/log607-2.txt
Registros: 6818
Prediccion: n^2
Tiempo: 1246.8 microsegundos
Comparaciones entre datos: 50624
Profundidad maxima de recursion: 14
Verificacion del orden: Correcto
Crecimiento teorico de Merge Sort: O(n log n)
Coincidio la prediccion con el crecimiento teorico: No
Esta evaluacion usa la complejidad teorica; una sola corrida no demuestra el crecimiento.

Primer registro ordenado:
Sep 08 2024 00:22:43 10.14.168.155 Unencrypted communication

Ultimo registro ordenado:
Sep 07 2026 23:50:40 10.14.178.169 Security breaches

Archivo generado: out/output608.out
```

El segundo archivo se ordenó con 50624 comparaciones, una profundidad máxima de 14 y un tiempo de 1246.8 microsegundos. Mi predicción fue un crecimiento cuadrático, por lo que no coincidió con el crecimiento teórico de Merge Sort. El programa confirmó el orden y generó el archivo de salida.

```
1  Sep 29 2024 10:00:00 10.0.0.1 Evento A primero
2  Sep 29 2024 10:00:00 10.0.0.2 Evento A segundo
3  Sep 30 2024 09:00:00 10.0.0.1 Evento A
4  Sep 30 2024 12:00:00 10.0.0.2 Evento B
5  Oct 01 2024 08:00:00 10.0.0.3 Evento C
6
```

Los dos eventos con la misma fecha y hora conservaron su orden de llegada después del ordenamiento.

Limitación de una prueba sugerida por IA y cómo se complementó

ChatGPT propuso comprobar los empates utilizando registros de prueba con claves numéricas y letras. Aunque esa prueba verificaba el algoritmo, por sí sola no comprobaba la lectura, la conversión de fechas y la escritura de logs completos. Al revisar qué demostraba la prueba, se identificó que faltaba comprobar el proceso completo. Para complementarla, se creó prueba_final.txt con dos eventos que tenían exactamente la misma fecha y hora y descripciones distintas. Después de cargarlos y ordenarlos, se revisó output608.out y se confirmó que "Evento A primero" permaneciera antes de "Evento A segundo".

Reflexión personal:

En esta evidencia pude relacionar las estructuras de datos de las actividades anteriores con una aplicación más completa. Antes había trabajado con listas, pilas y filas por separado. Al integrarlas en el procesamiento de logs, entendí mejor que cada estructura puede cumplir una función distinta dentro de un mismo programa.

La lista ligada simple sirvió como base de la pila y la fila. Uno de los cambios que más me ayudó a entender su funcionamiento fue agregar el apuntador tail. En mi versión anterior, insertar al final requería recorrer todos los nodos. Con este apuntador se puede llegar directamente al último nodo, aunque también hay que actualizarlo correctamente cuando se elimina un elemento o la lista queda vacía.

La fila permitió procesar las líneas en el orden en que llegaron del archivo. Primero se encolaron con su número de línea y después se desencolaron para validar sus datos. Guardar el número original permitió identificar dónde estaba cada error. Las líneas inválidas se omitieron y los registros válidos se agregaron al final de la lista doblemente ligada.

La lista doblemente ligada fue el almacén principal de los registros. Sus enlaces next y prev permiten recorrerla en ambas direcciones, pero también requieren cuidado al insertar, eliminar u ordenar nodos. Las pruebas de los extremos y de una lista con un solo elemento fueron útiles para comprobar que no quedaran conexiones incorrectas.

Para ordenar elegimos Merge Sort porque se adapta al trabajo con nodos y mantiene un crecimiento de O(n log n). En lugar de copiar los registros a un arreglo, divide la lista y une las mitades modificando sus conexiones. También conserva el orden de llegada

cuando dos registros tienen la misma fecha y hora, porque ante un empate selecciona primero el nodo de la mitad izquierda. Esto se comprobó con las claves y letras y después con logs completos.

La pila se utilizó para mostrar los resultados de una búsqueda en orden descendente. Aunque la lista doble ya permite retroceder con prev, la pila no era indispensable desde el punto de vista del funcionamiento: también se podría localizar el último registro del día y recorrer hacia atrás. Esa alternativa evitaría crear nodos adicionales. Sin embargo, la pila permitió aplicar directamente el comportamiento LIFO y cumplir con lo solicitado. Su desventaja es que necesita memoria adicional proporcional a la cantidad de resultados, aunque en este programa guarda apuntadores y no copias completas de los logs.

Otra diferencia respecto a la Evidencia 1 fue la búsqueda. En un vector se puede acceder directamente a una posición intermedia, pero en una lista ligada hay que recorrer nodos para llegar a ella. Por eso no se obtiene directamente la ventaja de O(log n) de la búsqueda binaria. Sería posible construir un arreglo de apuntadores a los nodos ordenados, pero requeriría recorrer la lista, ocupar memoria adicional y mantener ese índice actualizado. Para esta actividad utilizamos un recorrido que termina en cuanto pasa del día solicitado.

En las mediciones observé que el segundo archivo necesitó menos comparaciones: 50624 frente a 78330 del primero. También tardó menos en las corridas registradas, aunque el tiempo puede variar entre ejecuciones. La profundidad máxima fue de 14 en ambos porque tenían la misma cantidad de registros. Esa profundidad representa las llamadas de mergeSort que permanecen activas al mismo tiempo en la pila de llamadas del programa, que es distinta de la clase Stack utilizada para los resultados.

Mis predicciones iniciales no coincidieron con el crecimiento teórico del algoritmo. Esto me ayudó a distinguir entre el orden inicial de los datos, las comparaciones realizadas y la complejidad del algoritmo. Comparar dos archivos del mismo tamaño permite observar diferencias entre esas entradas, pero no demuestra por sí solo cómo crece el trabajo al aumentar la cantidad de registros.

ChatGPT tuvo una participación amplia en el desarrollo: revisó los códigos anteriores y generó pruebas. Trabajar por etapas me permitió revisar una parte antes de continuar con la siguiente. Sin ayuda de IA habría necesitado más tiempo para adaptar los apuntadores y consultar ejemplos, pero también habría tenido que escribir más partes por mi cuenta. En una siguiente actividad intentaría implementar primero funciones pequeñas y utilizar la IA para revisar mi intento. Además, debo respetar la separación entre las partes donde se permite asistencia y las que deben realizarse sin ella.